Lecture 7 — Spark I: Basics

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

1 What is Apache Spark?

Core insight: Apache Spark is a distributed computing framework that stores intermediate data in memory rather than writing it to disk, making it dramatically faster than Hadoop MapReduce — especially for iterative workloads where the same data is processed many times.

Think of Hadoop MapReduce as a factory assembly line that, after every step, boxes up the partially-finished product, ships it to a warehouse, and then retrieves it again for the next step. All that boxing, shipping, and unboxing is slow. Spark instead passes the partially-finished product directly to the next station in memory — no boxing, no shipping.

Distributed

Data and computation are split across many machines, allowing datasets too large for a single node to be processed in parallel.

In-Memory First

Intermediate results are kept in worker RAM. Disk is only used when memory is insufficient ("spill to disk").

Fault Tolerant

Lost data is recomputed using a **lineage graph** (DAG) rather than expensive replication.

2 Why Spark? Motivation over MapReduce

Hadoop MapReduce has two major limitations that Spark was designed to fix:

Problem 1 — Disk I/O Cost

Every MapReduce job writes its intermediate data (the output of the Map phase) to **local disk**, then reads it back for the Reduce phase. This is slow due to disk access times and network transfer during the shuffle.

Problem 2 — Not Iterative

Machine learning algorithms (e.g., gradient descent) iterate over the same dataset dozens or hundreds of times. In MapReduce, each iteration is a separate job: read from HDFS → compute → write to HDFS → repeat. This is extremely wasteful.

Spark's solution: keep data in memory across iterations. For a full-text search of 60 GB of Wikipedia on 20 EC2 machines, reading from cache took **0.5 seconds** vs. **20 seconds** from disk — a 40× speedup.

Spark also has dramatically simpler code. The WordCount program that takes ~60 lines in Hadoop MapReduce (with Mapper class, Reducer class, and a Driver class) is just **5 lines** in Spark.

3 Spark Architecture

Every Spark application has three core components working together:

Driver Process

Runs the user's `main()` function. Responds to user input, plans the computation, and distributes work to Executors. Receives final results.

↔

Cluster Manager

Allocates CPU and memory resources when the application requests them. Can be Spark's own manager, YARN, Mesos, or Kubernetes.

↔

Executors

Worker processes on each node. They execute the tasks assigned by the Driver and send results back. Each executor has a fixed number of CPU slots and an allocated chunk of memory.

In **local mode**, the Driver, Cluster Manager, and all Executors run on a single machine. This is useful for development and testing.

The API stack, from lowest to highest level, is: **RDDs** (raw distributed data) → **DataFrames / Datasets** (structured, SQL-like) → **Spark SQL, MLlib, GraphX, Structured Streaming** (domain-specific libraries built on top).

4 Evolution of Spark APIs

Spark has had three major API generations. Understanding all three is important because exam questions may use any of them.

RDDs (2011)

Resilient Distributed Datasets. The original Spark API. A collection of JVM objects distributed across machines. You use functional operators like `map`, `filter`, `reduce`. Low-level and powerful, but verbose.

No optimizer · Verbose

DataFrames (2013)

A collection of `Row` objects — think of it as a table with named columns, like a pandas DataFrame or SQL table. Uses an optimizer (Catalyst) that can automatically restructure queries for speed.

Optimizer · SQL support · Not type-safe

Datasets (2013)

Combines the best of both: type-safe (like RDDs) and fast with optimizer (like DataFrames). Internally uses rows; externally uses typed JVM objects. **Only available in Scala/Java**, not Python or R.

Type-safe · Optimizer

Exam trap: In Scala, `DataFrame` is simply a type alias for `Dataset[Row]`. They are the same thing. Datasets are **not available in Python** because Python is dynamically typed.

5 Resilient Distributed Datasets (RDDs)

An RDD is an immutable, distributed collection of objects. It represents data spread across many machines. Once created, it cannot be changed — you only ever *create new* RDDs from existing ones.

Creating an RDD: You can create an RDD by loading a file or parallelising a local collection:

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
# Creates an RDD with 4 strings split across 3 partitions (worker nodes)

textFile = sc.textFile("hdfs://...")
# Reads a file in parallel from HDFS; each worker reads a different block
```

5a — Transformations (Lazy)

A **transformation** takes one RDD and returns a new RDD. Critically, transformations are **lazy**: they do not actually execute when you call them. Instead, Spark records the transformation in a plan and waits.

```
dataRDD  = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
nameLen  = dataRDD.map(lambda s: len(s))   # records the plan — nothing runs yet!
```

Why lazy? Spark can look at the entire plan before executing. This lets it optimise the sequence of operations (e.g., push a `filter` earlier so less data needs to flow through later steps, or remove steps that are never needed).

`map` vs `flatMap`: `map` produces exactly one output element per input (1-to-1). `flatMap` produces a list per input and then flattens all lists into a single RDD (1-to-many). Use `flatMap` whenever one element should expand into multiple elements.

```
sentences = sc.parallelize(["Hello World", "Spark is great"])

# map: each line becomes a list — output is still nested
sentences.map(lambda line: line.split(" ")).collect()
# → [["Hello", "World"], ["Spark", "is", "great"]]

# flatMap: each line is split, then all words are merged into one flat RDD
sentences.flatMap(lambda line: line.split(" ")).collect()
# → ["Hello", "World", "Spark", "is", "great"]
```

`flatMap` is the first step in WordCount: every line of a text file is split into its words, producing many words per line, all flattened into a single stream that `map` and `reduceByKey` then operate on.

Common transformation operations:

Narrow Transformations

`map`, `flatMap`, `filter`, `contains`

Each partition of the *parent* RDD feeds into at most **one** partition of the child RDD. No data needs to cross the network.

Wide Transformations

`reduceByKey`, `groupBy`, `orderBy`, `join`

Each partition of the parent may contribute to **multiple** partitions of the child. Data must be shuffled across the network.

Operation	Type	Example	Result
map	Narrow	`rdd.map(lambda x: x * 2)`	`[2, 4, 6, 8, 10]`
flatMap	Narrow	`rdd.flatMap(lambda x: [x, -x])`	`[1,-1, 2,-2, 3,-3, ...]`
filter	Narrow	`rdd.filter(lambda x: x % 2 == 0)`	`[2, 4]`
reduceByKey	Wide	`pairs.reduceByKey(lambda a,b: a+b)`	`[("a",4), ("b",2)]`
groupByKey	Wide	`pairs.groupByKey()`	`[("a",[1,3]), ("b",[2])]`
sortBy	Wide	`rdd.sortBy(lambda x: x, ascending=False)`	`[5, 4, 3, 2, 1]`
join	Wide	`pairs.join(r2)`	`[("a",(1,"x")), ("a",(3,"x")), ...]`
select	DF only	`df.select("name", "age")`	DataFrame with 2 columns
filter	DF only	`df.filter(df["age"] > 25)`	Rows where age > 25
groupBy	DF only	`df.groupBy("dept").count()`	Count per department
orderBy	DF only	`df.orderBy("age")`	Sorted by age ascending
join	DF only	`df.join(df2, "id")`	Joined on "id" column

5b — Actions (Trigger Execution)

An **action** is the signal that tells Spark "run everything now and give me a result." Only when an action is called does Spark actually start computing.

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
nameLen = dataRDD.map(lambda s: len(s))   # transformation — still lazy
nameLen.collect()                          # action — NOW Spark executes!
# Output: [5, 3, 5, 6]
```

Different actions return different kinds of results. Here the same `flatMap` transformation is triggered by two different actions:

```
sentences = sc.parallelize(["Hello World", "Spark is great"])
words = sentences.flatMap(lambda line: line.split(" "))  # lazy — plan recorded

words.count()        # action — returns 5 (total words across all partitions)
# → 5

words.first()        # action — returns the first element only
# → "Hello"
```

Common actions: `collect()` (retrieve all elements to driver), `count()`, `first()`, `take(n)`, `save()`, `show()`.

Action	Example	Returns
collect()	`rdd.collect()`	`[1, 2, 3, 4, 5]` — all elements to driver
count()	`rdd.count()`	`5` — number of elements
first()	`rdd.first()`	`1` — first element only
take(n)	`rdd.take(3)`	`[1, 2, 3]` — first n elements
saveAsTextFile()	`rdd.saveAsTextFile("hdfs://out/")`	Nothing — writes partitions to file system

show(n)	`df.show(5)`	Nothing — prints first n rows to console (DF only)

5c — How Distributed Processing Works

When an action fires, Spark sends tasks to all workers in parallel. Each worker processes its own partition. Results are only sent back to the driver at the very end (for `collect()`) or aggregated incrementally (for `count()`).

1. Driver sends task instructions to each worker node.
2. Each worker applies the transformation(s) to its partition independently and in parallel.
3. Results from each partition are sent back to the Driver and combined.

> For `nameLen.collect()` on `["Alice","Bob","Carol","Daniel"]` split 3 ways: Worker 1 computes [5,3], Worker 2 computes [5], Worker 3 computes [6]. Driver collects and merges them into [5,3,5,6].

6 Caching — Reusing RDDs in Memory

By default, every time you call an action on an RDD, Spark recomputes it from scratch (re-reading the original data and replaying all transformations). For interactive or iterative workflows, this is very wasteful. **Caching** tells Spark to keep the RDD in worker RAM after it is first computed.

```
lines    = sc.textFile("hdfs://...")
errors   = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()                                        # pin this RDD in memory

messages.filter(lambda s: "mysql" in s).count()         # 1st query: loads from HDFS, caches
messages.filter(lambda s: "php" in s).count()           # 2nd query: reads from cache — FAST!
```

On the **first action**: Spark reads from HDFS, processes, and stores the result in each worker's memory. On **subsequent actions**: Spark reads directly from the in-memory cache — no disk I/O at all.

`cache()`

Stores the RDD in **memory only** on each worker node. This is a shorthand for `persist(MEMORY_ONLY)`. Fast to read, but if memory runs out, partitions are evicted and must be recomputed from scratch.

`persist(options)`

More flexible. Options include:
- `MEMORY_ONLY` — RAM only
- `MEMORY_AND_DISK` — RAM first, spill to disk
- `DISK_ONLY` — disk only
- Off-heap memory variants

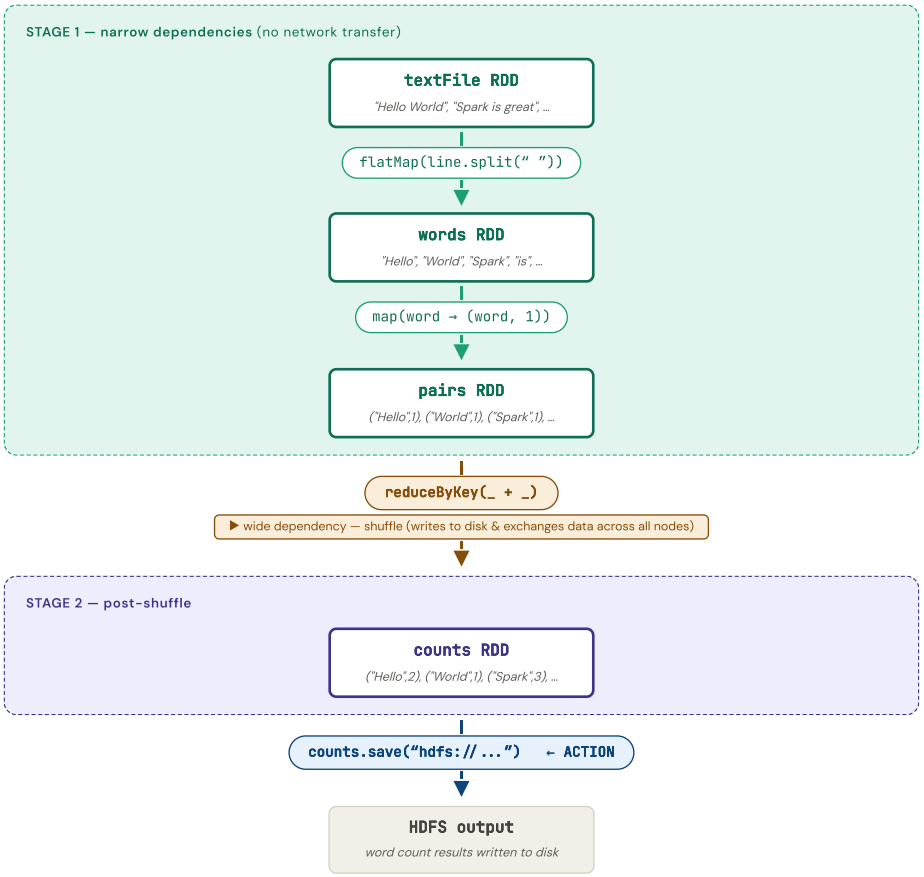
When to cache: Cache an RDD when it is (1) expensive to compute and (2) used more than once. Do not cache everything — memory is limited.

> **LRU eviction:** If worker nodes run out of memory, Spark evicts the **Least Recently Used** cached RDD partitions. The evicted partitions will be recomputed on demand from the lineage. Be aware of memory pressure when caching large datasets.

7 Directed Acyclic Graph (DAG) & Dependencies

Internally, Spark builds a **Directed Acyclic Graph (DAG)** to represent all the RDDs and how they are derived from one another. Every transformation adds a new node to this graph. Every action triggers Spark to evaluate the graph.

> **Transformations build the DAG. Actions trigger execution of the DAG.**



Spark uses the concept of **narrow** and **wide** dependencies to decide how to execute the DAG efficiently:

Narrow Dependency

Each partition of the *parent* RDD is used by **at most one** partition of the child RDD.

Examples: `map`, `flatMap`, `filter`

No shuffle needed — workers process independently.

Wide Dependency

Each partition of the parent RDD may contribute to **multiple** partitions of the child RDD.

Examples: `reduceByKey`, `groupBy`, `orderBy`

Shuffle required — data must be exchanged across machines.

Stages: Spark groups consecutive narrow-dependency transformations into a single **stage**. Within a stage, all operations run on the same machine without any network transfer. A new stage begins wherever a wide dependency (shuffle) is required.

Stage 1
Consecutive narrow transformations (e.g., textFile → flatMap → map). Run on each partition locally.

→

Shuffle Boundary
Wide transformation (e.g., reduceByKey). Intermediate results written to disk and exchanged across nodes — like MapReduce's shuffle.

→

Stage 2
Continues with further narrow transformations. Each partition now holds the merged data for a specific key.

Performance tip: Shuffles involve writing to disk and transferring data over the network — exactly what MapReduce does at every step. Minimising the number of wide/shuffle operations is one of the most important ways to optimise a Spark job.

8 Lineage & Fault Tolerance

Spark does **not** use replication (copying data to multiple machines) to handle faults the way HDFS does. Why? Because Spark tries to keep all data in memory, and memory is much more limited than disk. Duplicating everything in memory would be far too expensive.

Instead, Spark uses **lineage**: each RDD knows exactly how it was derived (which parent RDD and which transformation produced it). This information is stored in the DAG.

1. A worker node fails and the data in its partitions is lost.
2. Spark detects the failure and launches a replacement task on another worker.
3. Spark traces the DAG lineage to find the most recent ancestor of the lost partition that is still available (in cache, on disk, or as the original input file).
4. Only the lost partitions are recomputed — the rest of the job continues normally.

Key insight: Lineage-based fault tolerance is cheap because only the *lost partitions* need recomputation, not the entire dataset. Caching RDDs at intermediate stages makes recovery even faster by shortening the recomputation chain.

9 DataFrames

A **DataFrame** represents a table of data with named, typed columns — conceptually identical to a SQL table or a pandas DataFrame, but distributed across many machines. It is the **recommended** API for most Spark work.

DataFrames are a higher-level abstraction over RDDs. All DataFrame operations are ultimately compiled down to RDD operations by Spark's optimizer (Catalyst). You get convenience *and* performance.

Loading a DataFrame from CSV:

```
flightData2015 = spark\
    .read\
    .option("inferSchema", "true")\   # auto-detect column types
    .option("header", "true")\        # first row is column names
    .csv("/mnt/defg/flight-data/csv/2015-summary.csv")
```

Method 1 — SQL queries: Register the DataFrame as a temporary SQL view, then query it with plain SQL. The result is another DataFrame.

```
flightData2015.createOrReplaceTempView("flight_data_2015")

maxSql = spark.sql("""
    SELECT DEST_COUNTRY_NAME, sum(count) AS destination_total
    FROM flight_data_2015
    GROUP BY DEST_COUNTRY_NAME
    ORDER BY sum(count) DESC
    LIMIT 5
""")
maxSql.collect()   # action: runs the query and returns results to driver
```

Method 2 — DataFrame API: Chain transformation methods. This produces identical results to the SQL query above — Spark optimises both to the same execution plan.

```
from pyspark.sql.functions import desc

flightData2015\
    .groupBy("DEST_COUNTRY_NAME")\
    .sum("count")\
    .withColumnRenamed("sum(count)", "destination_total")\
    .sort(desc("destination_total"))\    # desc() returns a column object
    .limit(5)\
    .collect()
```

Transformation functions like `groupBy`, `sort`, `limit` are still **lazy** — they build a plan. Only `collect()` triggers execution.

10 Datasets

A **Dataset** is like a DataFrame but **type-safe**: instead of generic `Row` objects, it is parameterised with a specific class so the compiler can catch type errors at compile time rather than at runtime.

```
// Scala only — Datasets are not available in Python or R
case class Flight(DEST_COUNTRY_NAME: String, ORIGIN_COUNTRY_NAME: String, count: BigInt)

val flightsDF = spark.read.parquet("/mnt/defg/flight-data/parquet/2010-summary.parquet/")
val flights   = flightsDF.as[Flight]   // cast DataFrame to typed Dataset[Flight]
flights.collect()                       // returns Array[Flight], not Array[Row]
```

> **Python note:** Python does not have Datasets because it is dynamically typed — type checking happens at runtime, not compile time. In Python, you only use RDDs and DataFrames. In Scala, `DataFrame = Dataset[Row]`, so they are the same type.

11 RDD vs DataFrame vs Dataset

Dimension	RDD	DataFrame	Dataset
Abstraction level	Low-level	High-level	High-level
Data representation	Any JVM objects	`Row` objects	Typed JVM objects
Type safety	Yes (compile time)	No (runtime)	Yes (compile time)
Catalyst optimizer	No	Yes	Yes
SQL interface	No	Yes	Yes
Languages	Scala, Java, Python, R	Scala, Java, Python, R	Scala, Java only
When to use	Low-level control; unstructured data	Most tasks; best for Python users	Type safety needed in Scala/Java
Introduced	2011	2013	2015 (Spark 1.6)

Spark vs Hadoop MapReduce

Dimension	Hadoop MapReduce	Apache Spark
Intermediate storage	Always writes to HDFS (disk)	In-memory by default; spills to disk only if needed
Iterative workloads	Very slow — each iteration re-reads from disk	Fast — data cached in memory across iterations
Fault tolerance	Replication (3× default on HDFS)	Lineage recomputation via DAG
Programming model	Must express everything as map + reduce	Rich API: map, filter, join, groupBy, SQL, ML, etc.
Ease of use	Verbose — many lines of boilerplate	Concise — WordCount in 5 lines
Speed (iterative)	Baseline	Up to 100× faster in memory

12 Exam-style Q&A

Q: What is the difference between a transformation and an action in Spark? Give one example of each.

> A transformation takes an RDD and returns a new RDD, but is *lazy* — it is not executed immediately. Example: `map()`, `filter()`. An action triggers the actual computation and returns a result to the driver or writes output. Example: `collect()`, `count()`. Spark only runs the computation when an action is called, which allows it to optimise the full execution plan first.

Q: Explain the difference between narrow and wide dependencies in Spark. Why does this distinction matter?

> A narrow dependency is where each partition of the parent RDD contributes to *at most one* partition of the child (e.g., `map`, `filter`). A wide dependency is where a parent partition may contribute to *multiple* child partitions, requiring a shuffle (e.g., `reduceByKey`, `groupBy`). The distinction matters because: (1) Spark groups consecutive narrow dependencies into a single "stage" executed on the same machine with no network transfer, and (2) wide dependencies require shuffling data across the network and writing to disk, like MapReduce. Minimising wide dependencies improves performance.

Q: How does Spark achieve fault tolerance without replicating data?

> Spark uses **lineage**. Each RDD records how it was derived from its parent(s) in a DAG. If a worker fails and its partitions are lost, Spark uses the lineage to recompute only the lost partitions from the nearest available ancestor (the original input file, a checkpoint, or a cached RDD). This avoids the need for expensive 3× replication in memory. Replication on disk (HDFS) is still used for the original input data.

Q: When should you use `cache()`? What happens if you cache too many RDDs?

> Use `cache()` when an RDD is (1) expensive to recompute and (2) will be used more than once. If too many RDDs are cached and memory runs out, Spark applies LRU (Least Recently Used) eviction — the least recently accessed partitions are dropped from memory and must be recomputed from lineage when needed again. Caching indiscriminately can slow things down by causing constant eviction and recomputation.

Q: What is a DAG in Spark? What builds it and what triggers it?

> A DAG (Directed Acyclic Graph) is Spark's internal representation of the computation plan — a graph of RDD nodes connected by transformations. **Transformations build the DAG** (by adding nodes) but execute nothing. **Actions trigger execution** of the DAG, causing Spark to evaluate the full graph, optimise it, and run the tasks on the cluster.

Q: What is the key advantage of DataFrames over RDDs?

> DataFrames go through Spark's Catalyst optimiser, which can automatically restructure the query for better performance (e.g., pushing filters earlier, selecting only needed columns). RDDs have no such optimiser — Spark runs exactly what you write. DataFrames also provide a SQL interface and are easier to use. The trade-off is that RDDs offer more low-level control for operations that do not fit the structured/tabular model.

Q: Why are Datasets not available in Python?

> Datasets provide compile-time type safety, which requires a statically typed language where the type of every variable is known before the program runs. Python is dynamically typed — types are checked at runtime, not compile time. Therefore, the type-safety guarantee that Datasets provide cannot be enforced in Python, and Datasets are only available in Scala and Java.